

Contents

1	Machine learning and artificial intelligence	2
1.1	Prediction	2
1.1.1	Regression	2
1.1.2	Classification	2
1.2	The supervised machine learning process	2
2	k-nearest neighbours (kNN)	3
2.1	Euclidean distance measure	3
3	Training the model	3
3.1	Train-test data split	4
3.2	Fitting the model (<code>train()</code> from <code>caret</code> package)	5
3.2.1	Error metrics - how a model is evaluated	5
3.2.2	Extra: other evaluation metrics	6
3.2.3	Cross-validation	6
4	Testing the model	7
4.1	Evaluating the final model	7
5	Training a reduced model	8
5.1	Extra: visualising the kNN model	9
6	Extra: Leave-One-Out Cross-Validation (LOOCV)	10
7	Extra: Custom values of k	10
8	Limitations	11
9	Conceptual Check	11

1 Machine learning and artificial intelligence

The previous weeks have been about dataset management with `dplyr` and understanding data graphically with `ggplot2`. The next weeks will focus on the name of the course - machine learning and artificial intelligence.

There are several types of machine learning that we will see in this course. For now, we will focus on using models and data for **prediction**. Other machine learning methods may not have a target to predict and instead focus on learning about the data.

1.1 Prediction

Prediction is a form of **supervised learning**, in which we wish to create a machine learning model to **predict or model** some *outcome variable* using some *predictor variables*. In this course, we will learn about two types of prediction.

1.1.1 Regression

Regression is used to model and predict some *numerical variable* based on data. For example, we may try to predict a student's exam result (0 to 100) based on how many hours they studied (more study time often correlates with higher scores). In this case, the exam result is the outcome variable, and we only have one predictor variable, the hours of study.

1.1.2 Classification

Classification is used to model and predict some *categorical variable* based on data. An example we will see later in the course is predicting whether a tumor is "benign" or "malignant" using scan results or blood tests.

More abstract and difficult examples of classification include processing other forms of data, including language and images, which we will not cover in this course. For example, we may try to classify whether an email is "spam" or "not spam" based on keywords, sender info, or links. Another example is identifying if a photo contains a "cat" or "dog" by analysing pixels and shapes.

1.2 The supervised machine learning process

When we have a target variable we are trying to predict, the process is largely the same:

1. We have a dataset. We split it into a **training dataset** and a **testing dataset**.
2. Train the model with the **training dataset** to get the best parameters for the model.
3. Evaluate the model with the **testing dataset** to see how the model performs on unseen data.

You will see this process over and over in the next few weeks.

2 k-nearest neighbours (kNN)

k-nearest neighbours is a simple machine learning algorithm, which can be used for **both** regression and classification. For now, we will see how it is used in regression, and then its use in classification in a few weeks. In both cases, the general principle is the same.

The algorithm behind kNN is simple. Say that we have some existing dataset of students' exam results and their time spent studying. To predict how well a student does with 10 hours of study time, the kNN algorithm then finds the k 'nearest' student(s) in its dataset, and predicts the grade of that student with 10 hours of study time based on the k 'nearest' student(s).

Within our kNN model, there is only **one parameter** that we need to choose to make the 'best' model possible: how many of the 'nearest' students should we take into account to make our prediction, i.e. what k value should we use? Theoretically, we could just choose any number for k and call it a model, but we will use our data to answer this question.

This example will be made clear in the tutorial example. For now, we should define what it means to be for a student to be 'near' to our new student. Later, we will see how we can choose the 'best' model.

2.1 Euclidean distance measure

Focusing solely on continuous, numerical variables, we can use a very common notion of distance. While it is referred to as the Euclidean distance measure in the tutorial notes, it is just our everyday notion of distance, used in Pythagoras' theorem, i.e.

$$a^2 + b^2 = c^2 \longleftrightarrow \sqrt{a^2 + b^2} = c$$

This measurement of distance d extends to as many dimensions/features $x_1, \dots, x_n$ as we want:

$$d^2 = x_1^2 + \dots + x_n^2 \longleftrightarrow d = \sqrt{x_1^2 + \dots + x_n^2}$$

Using this measure, it is important that all our measurements are then on the same scale. For example, while the size of a house in square feet may have a range of 1500 square feet, the number of rooms may only have a range of 5 rooms. The effect of the difference in the size will then dominate any effect of the difference in the number of rooms that the number of bedrooms is rendered useless.

Scaling is then required - there are several possible ways to scale the data for each to match, which we will see in future weeks. For now, understand that scaling the data is important due to this. The function we will use this week to train our kNN model will do this scaling for us.

3 Training the model

The training process for all models in a machine learning context is extremely similar. Essentially, it boils down to selecting the parameters of the model. For the kNN model, the training process is

to select the 'optimal' k . Following the tutorial notes, we will use the Boston dataset in the MASS package.

```
data("Boston", package="MASS")    # an alternative way to import data from
↪ packages
```

For now, perform an inspection of the data to understand what it contains. You may find more details about each variable by checking its documentation. We will be trying to predict the medv variable, which represents the median value of a home, with our kNN model.

```
str(Boston)
?Boston
```

3.1 Train-test data split

In all machine learning processes, we require data to train a model (training data). We also require data to test a model (testing data). The same dataset cannot be used for both training and testing. The test data is meant to give an unbiased measurement of how the model performs on data it has never seen before, so mixing the two or reusing training data as test data is a bad practice (in general).

Since we only have a single dataset to use for both training and testing, a common practice is to simply *segment the data* you have into two sets, one for training (80%), and one for testing (20%). There are no hard rules on the proportion of this split, but in general, your training data should be larger than your test data.

To split the data in R, we use the `sample()` function in base R to randomly select which observations to include in the training or test set. Since there is an element of randomness, to make our runs repeatable and to remove the randomness, we can use `set.seed()` to define how numbers are chosen. It is not important in this course how the numbers are chosen, but this involves *pseudorandomness*. **Without `set.seed()`, it is unlikely or impossible for any results you have that involve randomness to be replicated.**

The first parameter of `sample()` indicates the set to sample from, and the second indicates how many to sample. In this case, we want to sample from numbers 1 to 506 (`nrow(Boston)`), and only 80% of them.

```
# you can choose any number.
# the number you choose determines the random numbers you will draw.
set.seed(100)
train.idx <- sample(nrow(Boston), nrow(Boston)*0.8)
cat("train.idx has", length(train.idx), "entries. head:", head(train.idx),
↪ "\n")
train.data <- Boston[train.idx, ]
test.data <- Boston[-train.idx, ]
dim(train.data)
dim(test.data)
```

```
## train.idx has 404 entries. head: 202 503 358 112 499 473
## [1] 404 14
## [1] 102 14
```

3.2 Fitting the model (`train()` from `caret` package)

For kNN, we will be using the `caret` package. Install and import it as needed.

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

Note that we use `set.seed()` here, as the training of this model involves some randomness.

```
set.seed(101)
model <- train(medv ~ ., data = train.data, method = "knn",
  trControl = trainControl("cv", number = 10),
  preProcess = c("center", "scale"),
  tuneLength = 10)
```

There are several parts of this function to understand:

`medv ~ .` indicates that we want to model the `medv` variable using every other variable in the dataset as a predictor, indicated by the dot `.`. Of course, we also have to specify the dataset, which is done with the `data` parameter. Likewise, `method = "knn"` indicates we want to train a kNN model.

`trControl = trainControl("cv", number = 10)` indicates how we want the training process to be done. "cv" means cross-validation, which is a method of helping us choose the parameters in a model.

`preProcess = c("center", "scale")` tells `train()` how the data should be pre-processed before the training starts. "center" will shift the mean of the data to 0 (for numerical stability), while "scale" will transform every predictor data feature to have a standard deviation of 1 (normalisation of scales between predictors). Having a standard deviation of 1 across all our predictor variables makes them (more) comparable using the Euclidean distance measure, so that one feature will not have an dominating effect when comparing distances.

`tuneLength = 10` simply indicates how many values of k we would like to try. Here, 10 values will be tested, in intervals of 2.

The trained kNN model is then saved to a variable `model`.

3.2.1 Error metrics - how a model is evaluated

Error metrics give us a numerical measure of how well a model performs, and as a basis of comparison against other models (given the same train and test data). There are many different error metrics, and

in this tutorial, given that we are predicting a *continuous numerical variable*, we used the **Root Mean Squared Error** (RMSE). This is computed by taking all observations in the test set, using its features x_i to predict the outcome variable $\hat{y}_i$, and calculating the squared difference from the true outcome value y_i . The mean of all these squared differences is the RMSE.

More mathematically, writing it out for all our observations (x_i, y_i) and our predictions $\hat{y}_i$ we have:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

The lower this RMSE value, the closer the model's predictions were to the true values, i.e. it was better at predicting the outcome based on the features. In the above graph of the RMSE against the number of neighbours k , we achieved a minimum at $k = 7$.

3.2.2 Extra: other evaluation metrics

The RMSE is not the only evaluation metric that we can use. As long as it gives us a numerical representation of how well the model performs, it is ok! In this case, we are predicting a *number* and comparing how close our predictions are.

Some common options include the mean absolute error (MAE) and the mean squared error (MSE). Broadly speaking, *squaring the error* penalises bad predictions more. The RMSE is also *on the same scale as the data* (since we square-root a square), unlike the MSE.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

3.2.3 Cross-validation

The objective of training our kNN model is the selection of one parameter, i.e. the number of neighbours k . The selection of this parameter k must be done using **only the training data**. To evaluate each possible value of k , we will need another test set to use *only in the training process*, separate from the test set *used for evaluating the trained model*, which we will call the *validation set*.

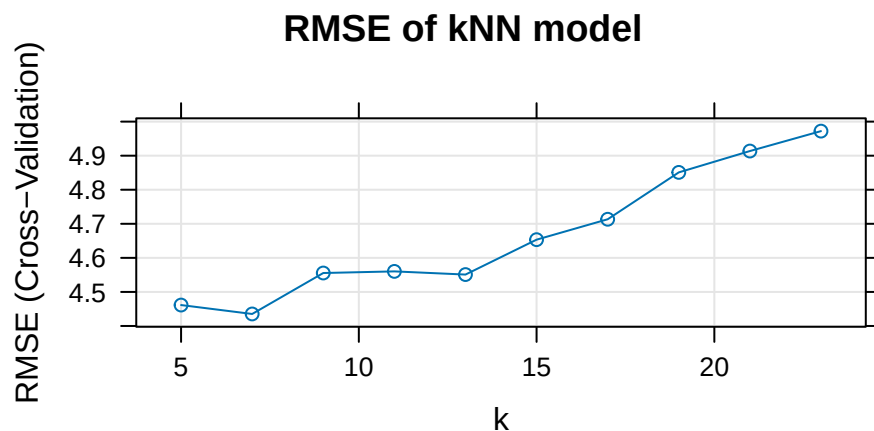
Cross-validation is a method to test models in the training process. From above, we have number = 10 in the training control parameter. This will randomly split the **training data** into 10 separate parts, called *folds*. Since there are elements of randomness, use `set.seed()` to ensure your results can be replicated. Call our 10 folds of the training data $d_1, \dots, d_{10}$.

For each value of k to be tested, 10 models m_i^k will be built, each excluding the i th part, which will be used as a validation set. For example, if we are testing $k = 7$, model m_3^7 will use $d_1, d_2, d_4, d_5, \dots, d_{10}$ as the training set, and d_3 to evaluate the trained model. The results of all models m_i^7 are then averaged to get how well that value $k = 7$ worked on our training data.

The model building and testing is done for every other value of k to be tested, giving us the following graph. The 'best' model, which is described in the following section, is chosen, and all the data use

used to create the model.

```
plot(model, main="RMSE of kNN model", xlab="k")
```



```
model$bestTune
```

```
## # A tibble: 1 x 1
##       k
##   <int>
## 1     7
```

For this course, the guideline is to have 40-50 samples in each fold. Realistically, there are no hard and fast rules - you can even have a single sample in each fold (i.e. a validation set of one observation), which is known as Leave-One-Out Cross-Validation (LOOCV). However, do note that the more folds, the more models must be trained, and so the time to train the model will increase.

4 Testing the model

Testing a model evaluates its use on **data that the model has never seen before**. In the kNN model's training above, we used one *metric*, the RMSE, to measure how the trained model performs against its validation set. We will also conduct a final testing on the test data that we segmented at the very beginning.

4.1 Evaluating the final model

The RMSE graph we obtained above was calculated using only the *validation sets* in the *training data*. Based on that, we determined $k = 7$ to be the best choice, and will use it in our trained model. To finally determine the performance of our model, we must apply the model to **data the model has never seen before**, i.e. the *test set*.

We will compute the RMSE of the test set with the built-in `predict()` function, and the `RMSE()` function from `caret`. Remember that our model was built to predict the `medv` value, so we compare our predicted values with the true values from the test data.

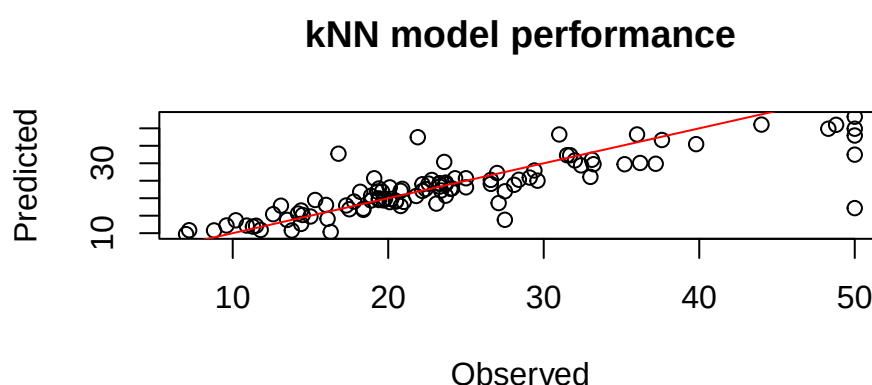
```
yhat <- predict(model, test.data)
y <- test.data$medv
head(yhat)      # first 6 predictions
head(y)         # and the corresponding real values
rmse.all <- RMSE(yhat, y)
rmse.all
```

```
## [1] 30.08571 18.61429 19.35714 16.84286 15.20000 17.00000
## [1] 36.2 27.1 18.9 17.5 14.5 18.4
## [1] 5.714944
```

The RMSE, as you can see, is significantly higher than what we saw during the training process with cross-validation. Generally this is what we will observe. Sometimes, the RMSE will be lower, but that is just a fact of the dataset and chance.

We can also plot the observed values against the predicted values. A diagonal line $y = x$ is added, indicating that if a point plotted lies on that line, the model predicted the outcome exactly for that observation. Any point deviating from that line was inaccurate by some amount. The further away from that line, the less accurate the model was at predicting the outcome value for that observation.

```
plot(test.data$medv, yhat, main="kNN model performance", xlab="Observed",
     ↪ ylab="Predicted")
abline(0,1, col="red")      # plot the line y = x
```



5 Training a reduced model

Above, we built a model using all 13 predictors. We will compare how it performs to a model that uses only 1 or 2 predictors, keeping the same train and test data. In this case, we will use `age` and `dis` as

the predictors.

```
set.seed(101)
model.1 <- train(medv ~ age, data=train.data, method="knn",
  trControl=trainControl("cv", number=10),
  preProcess=c("center", "scale"),
  tuneLength=10)
yhat.1 <- predict(model.1, test.data)
rmse.1 <- RMSE(yhat.1, test.data$medv)

set.seed(101)
model.2 <- train(medv ~ age + dis, data=train.data, method="knn",
  trControl=trainControl("cv", number=10),
  preProcess=c("center", "scale"),
  tuneLength=10)
yhat.2 <- predict(model.2, test.data)
rmse.2 <- RMSE(yhat.2, test.data$medv)

cbind(rmse.1, rmse.2, rmse.all)
```

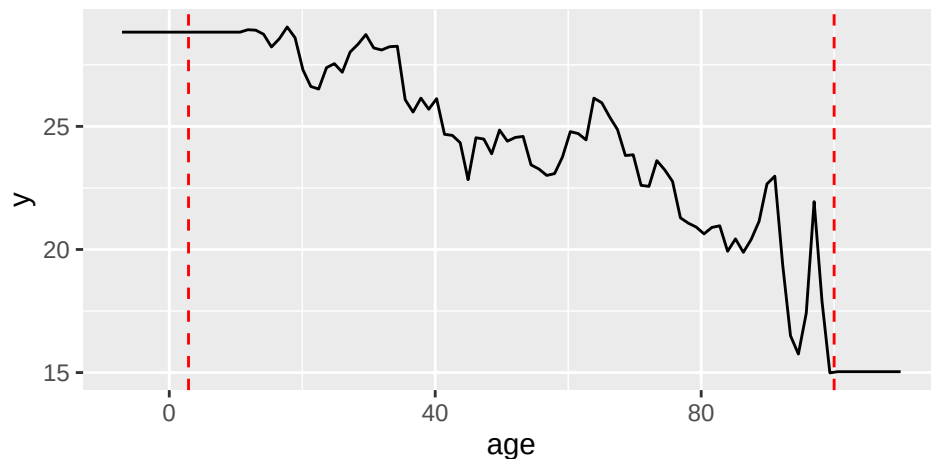
```
##          rmse.1   rmse.2 rmse.all
## [1,] 10.26291  9.715167  5.714944
```

We can see that the model with only one predictor performed not as well.

5.1 Extra: visualising the kNN model

We can visualise the model with one predictor. This graph shows us for **any value** of age input, the value of medv that the model outputs. age has a range of 2.9 to 100 in train . data. The red dashed lines indicate the minimum and maximum values of age in the train data.

```
x <- data.frame(age=seq(-7.1, 110, length.out=100))
y <- predict(model.1, x)
ggplot(cbind(x, y), aes(age, y)) + geom_line() +
  geom_vline(xintercept=c(2.9,100), color="red", linetype="dashed")
```



6 Extra: Leave-One-Out Cross-Validation (LOOCV)

Using cross-validation, we can have as many folds as data points in the training dataset. The model will take longer to train due to the number of models trained and evaluated. In this case, we get the same final value of k , resulting in the same model.

```
model.loocv <- train(medv ~ ., data=train.data, method="knn",
  trControl = trainControl("cv",
    ↪ number=nrow(train.data)),
  preprocess = c("center", "scale"),
  tuneLength = 10)

yhat.loocv <- predict(model.loocv, test.data)
rmse.loocv <- RMSE(yhat.loocv, test.data$medv)
rmse.loocv
```

7 Extra: Custom values of k

Using tuneLength will test for $k = 3, 5, 7, \dots$, the odd numbers. To test specific values of k , we can use the tuneGrid parameter, passing in a dataframe containing the parameters to test.

```
model.tuneGrid <- train(medv ~ ., data=train.data, method="knn",
  trControl = trainControl("cv", number=10),
  preprocess = c("center", "scale"),
  tuneGrid = data.frame(k=c(15, 19, 3)))

model.tuneGrid
```

```
## k-Nearest Neighbors
##
## 404 samples
```

```
## 13 predictor
##
## Pre-processing: centered (13), scaled (13)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 364, 364, 363, 364, 363, 363, ...
## Resampling results across tuning parameters:
##
##    k    RMSE      Rsquared    MAE
##    3  4.213730  0.7805694  2.735102
##    15  4.750622  0.7518052  3.093476
##    19  4.872929  0.7416180  3.194724
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was k = 3.
```

8 Limitations

The kNN model's predictive power is fully based on the data it has seen before. The model, in fact, only consists of **the training data**. Thus, it is impossible to extrapolate results that have features far away from our training dataset.

kNN is a **non-parametric** model, where we do not suppose any parameterisation between the outcome variable and predictor variables. In non-parametric models, there is also little insight into the effects of each predictor variable on the outcome variables. Next week, we will see a parametric model, linear regression.

9 Conceptual Check

1. What is the aim of the training process? What is the aim of the testing process?
2. Why is it important to have a testing dataset, distinct from the training dataset?
3. What does "Euclidean distance" mean? How is it calculated?
4. Explain briefly how a kNN model works. How does a kNN model predict values?
5. Is the scale/spread of predictors important in kNN? Why or why not? If it is important, give an example of when it is important.
6. How many kNN models are trained and evaluated in a 10-fold cross-validation, with 6 different values of k?